

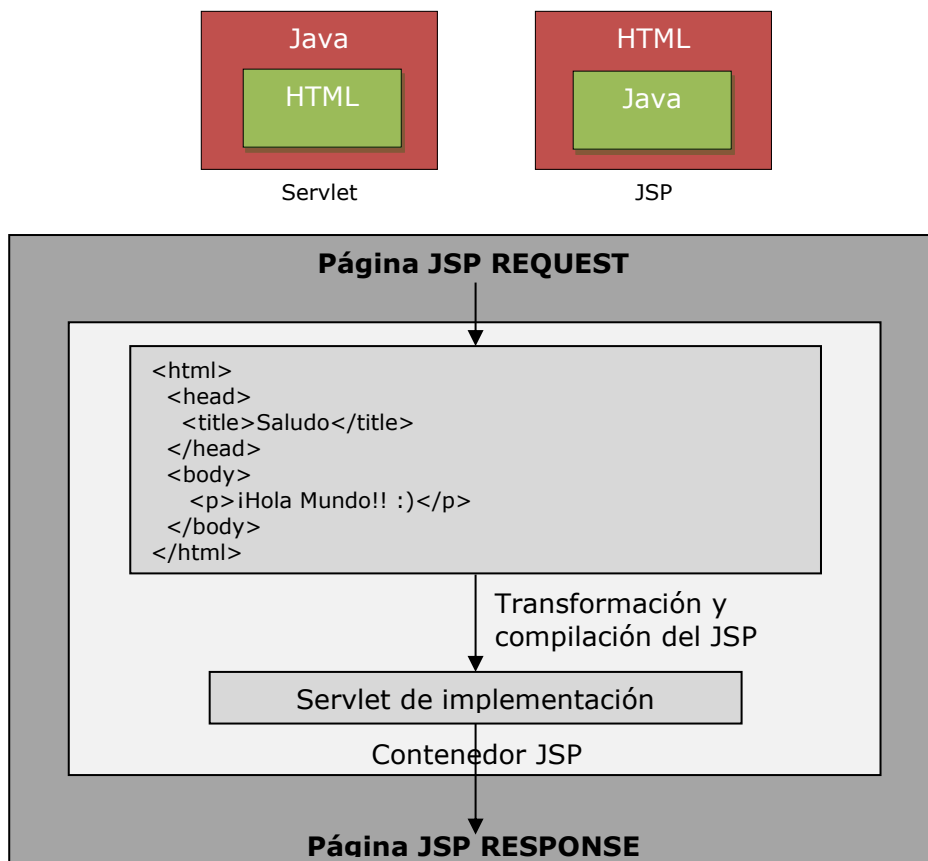
 UNIVERSIDAD Blas Pascal	INGENIERÍA EN INFORMÁTICA – PLAN 2014 PROGRAMACIÓN DISTRIBUIDA Y COMPONENTES – 9º CUATRIMESTRE	
	APUNTE DE JSP Y JAVA BEANS	VERSIÓN: 1.5 VIGENCIA: 09-04-2018

JSP (Java Server Page)

Jakarta Server Pages, o formalmente, Java Server Pages (JSP) es una tecnología que ayuda a los desarrolladores a crear páginas Web dinámicas basadas en HTML o XML en aplicaciones Web de Java.

La tecnología JSP es una extensión de la tecnología Servlet, es implementada a través de la interfaz `jakarta.servlet.Servlet` de acuerdo con la especificación Servlet 6.0. Por lo general, las páginas JSP se usan cuando la mayor parte del contenido que se envía al cliente es texto estático, y sólo una pequeña parte de ese contenido se genera dinámicamente con código Java. Por su parte, los Servlets se usan en el caso inverso, es decir, cuando una pequeña porción del contenido a enviar es texto estático. De hecho, muchos Servlets no producen contenidos, y en vez de ello realizan alguna tarea y luego invocan a otro Servlet o a una página JSP para proveer una respuesta al cliente.

Si programáramos un Servlet y una página JSP que generaran el mismo código HTML, veríamos que en un Servlet el HTML es generado desde el código Java, mientras que en un JSP el código Java es embebido en el código HTML.

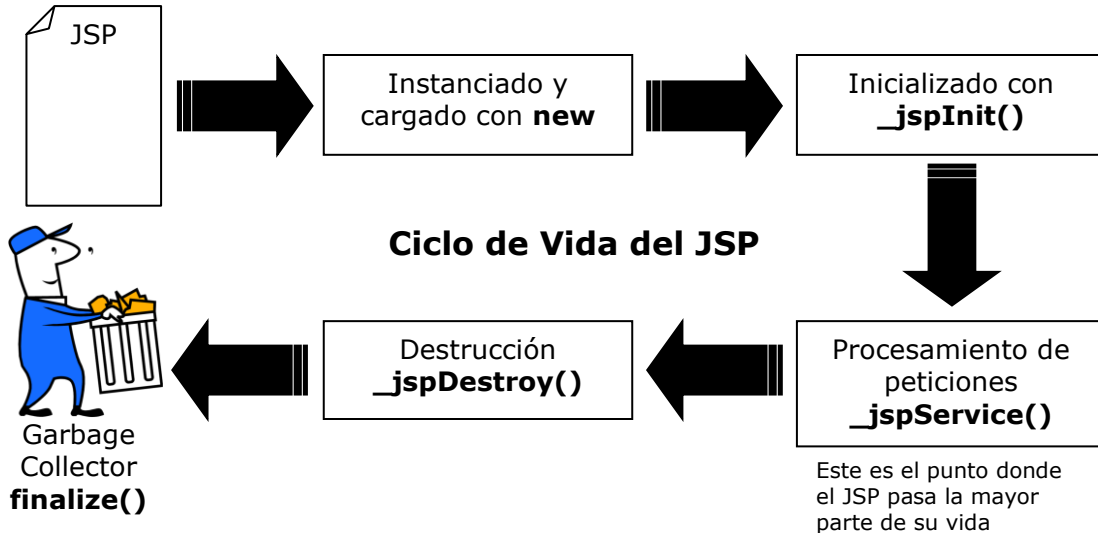


Ciclo de Vida del JSP

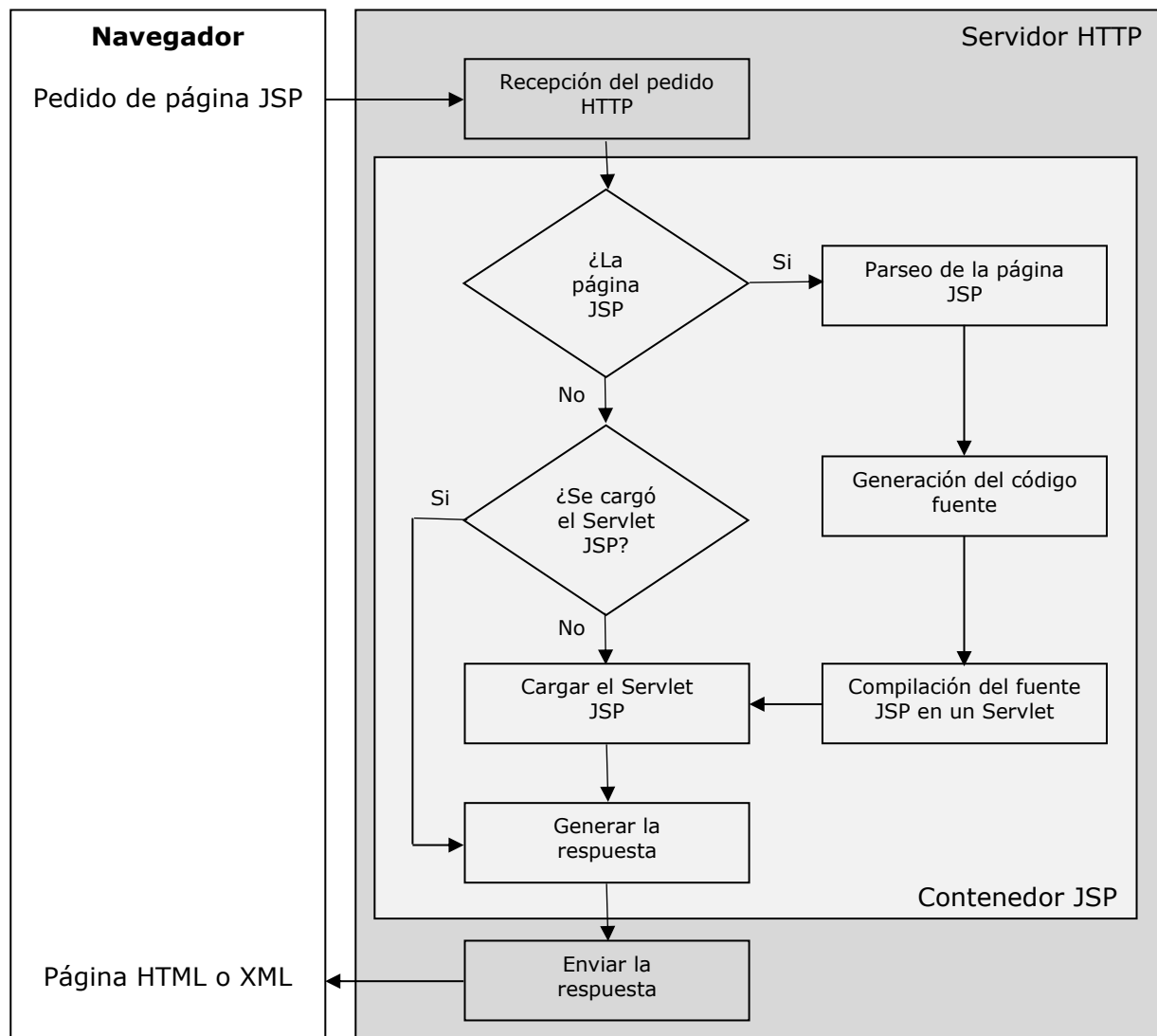
El contenedor JSP es el encargado de regular y controlar el ciclo de vida de un JSP. El ciclo de vida consiste en cuatro fases:

- *Traslación:* Si la sintaxis es correcta se traslada al Servlet de implementación.
- *Inicialización:* El contenedor JSP carga el Servlet de implementación y crea una instancia del Servlet para procesar el pedido. Se invoca al método `_jspInit()`
- *Ejecución:* Después de que el contenedor cargó e inicializó el Servlet de implementación, el pedido es procesado. Para resolver el pedido se llama al método `_jspService()`

- **Finalización:** El contenedor invoca el método `_jspDestroy()`. Después de que este método es invocado el Servlet de implementación no puede responder a ningún otro pedido.



¿Cómo trabaja una página JSP?



 UNIVERSIDAD Blas Pascal	INGENIERÍA EN INFORMÁTICA – PLAN 2014 PROGRAMACIÓN DISTRIBUIDA Y COMPONENTES – 9º CUATRIMESTRE	
	APUNTE DE JSP Y JAVA BEANS	VERSIÓN: 1.5 VIGENCIA: 09-04-2018

Es común que un proceso JSP se designe como scriptlet, lo cual ayuda a distinguir del concepto de script común.

Habitualmente, un programa JPS se compone de una combinación de etiquetas HTML y de etiquetas JSP. Estas últimas definen el código Java que se ejecutará antes de enviar la salida al navegador. Las etiquetas JSP comienzan con la secuencia **<%** y finalizan con la secuencia **%>**. Hay también una versión con formato XML para las etiquetas JSP: estas etiquetas comienzan con **<jsp:etiqueta>** y finalizan con **</jsp:etiqueta>** y donde la palabra etiqueta se reemplaza con alguna palabra específica. Ambas formas de etiquetas se incluyen o entremezclan dentro del HTML, en forma similar a como lo hacen los códigos de los lenguajes de script.

El código Java contenido dentro de una etiqueta JSP debe ser compilado en algún momento. La compilación es realizada por una aplicación generalmente llamada "motor de JSP" o "intérprete de JSP". Un motor de JSP muy conocido es Apache Tomcat.

Técnicamente, la página JSP es compilada por el motor de JSP sólo la primera vez que esa página es invocada. En ese momento, el motor crea un Servlet a partir de la página, y lo ejecuta. El resultado de la ejecución es enviado al navegador cliente que solicitó la página. De allí en más, cada vez que la página se invoca se ejecuta el Servlet directamente. La página sólo necesita ser recompilada cuando sufra alguna modificación en su contenido.

Las etiquetas JSP pueden ser de cinco tipos diferentes:

- **Comentarios:** Comienzan con **<%--** y finalizan con **--%>**. Contienen comentarios o aclaraciones de texto, que son ignorados por el motor al compilar:

```
<%-- Ejemplo de comentario JSP --%>
```

- **Declaraciones:** Comienzan con **<%!** y finalizan con **%>**. Contienen instrucciones Java que declaran variables, objetos o métodos que luego pueden ser usados en cualquier otro lugar de la página JSP. Las variables declaradas de esta forma son variables de instancia que se mantendrán cuando la página vuelva a llamarse. Es importante recordar que cuando la página JSP se compila por primera vez, el motor crea un objeto Servlet, y toda variable o método declarado en la página mediante la etiqueta **<%!** será un atributo o un método de ese Servlet:

```
<%! int contador = 0; %>
```

- **Directivas (o acciones):** Comienzan con **<%@** y finalizan con **%>**. Le indican al motor de JSP que realice alguna tarea o acción determinada, como puede ser importar un paquete de Java (entre otras acciones). Dentro de esta forma de etiqueta, aparecen parámetros diversos, dependiendo de la acción que se quiere llevar a cabo. Los tres tipos de directivas son las siguientes:

- o **page:** permite configurar la página con valores y elementos que serán útiles al proceso. Por ejemplo,

```
<%@ page import="java.util.*" %>
```

- o **include:** permite insertar otros recursos dentro de la página (tal como código fuente incluido en otro archivo). Por ejemplo,

```
<%@ include file="../docs/ejemplos.html" %>
```

 UNIVERSIDAD Blas Pascal	INGENIERÍA EN INFORMÁTICA – PLAN 2014 PROGRAMACIÓN DISTRIBUIDA Y COMPONENTES – 9º CUATRIMESTRE	
	APUNTE DE JSP Y JAVA BEANS	VERSIÓN: 1.5 VIGENCIA: 09-04-2018

- **taglib**: permite que un programador incluya sus propias librerías de tags, para simplificar el código JSP. Por ejemplo,

```
<%@ taglib uri="/WEB-INF/tld/etiquetas.tld" %>
```

- **Expresiones**: Comienzan con **<%=** y finalizan con **%>**. Contienen una expresión, cuyo resultado reemplazará a la etiqueta al procesar la JSP. El valor de la expresión será mostrado en la página HTML que se presente al cliente en el navegador. Sólo las etiquetas que contengan expresiones de este tipo serán transformadas y mostradas al cliente al devolverle la página. Estos elementos son los que permiten que finalmente la página tenga contenidos no fijos, actualizables según los datos de cada cliente:

```
<% int contador = 0; %>
<p>Contador: <%= contador %></p>
```

- **Scriptlets (o procesos)**: Comienzan con **<%** y finalizan con **%>**. Contienen bloques de instrucciones Java que conforman un proceso a realizar en la página:

```
<%
contador++;
out.println("<p>Contador: " + contador + "</p>");
%>
```

Como se dijo, una página JSP se usa normalmente para permitir interacción entre los usuarios y el sistema Web. Los usuarios cargan páginas en las que se les solicita enviar datos, mediante algún formulario HTML consistente en campos de edición, casillas de selección, botones, y otros elementos. Cuando el usuario carga todos los datos pedidos, generalmente se dispone de un botón para enviar esos datos al servidor de la aplicación, y si se presiona ese botón se recogen todos los datos del formulario y se envían al servidor en alguna forma que depende del valor del atributo **method** usado al definir el formulario. Ese valor puede ser "get" o "post" (si no se especifica, el valor default es get):

```
<form method="post" action="response.jsp">
</form>
```

El otro atributo del formulario es **action**, y se usa para indicar cuál es el programa del servidor que recibirá y procesará el envío al presionar el botón de enviar. Normalmente se tratará de una página JSP o de un Servlet. En el código anterior, se supone que ese programa será una página JSP llamada response.jsp.

El programa de respuesta (indicado en el atributo action) debe ser capaz de tomar los valores enviados, procesarlos y generar la página de respuesta. Además de la programación en Java propia de la JSP, toda JSP dispone de una serie de objetos predefinidos y ya creados, listos para usar en diversas tareas específicas. Uno de esos objetos es el objeto **request** (de la clase `HttpServletRequest`), que se usa para acceder a los valores de los campos enviados en la "cadena de petición" o "query string". Dispone de un método **getParameter()** que toma como parámetro una cadena con el nombre del campo que se quiere acceder, y retorna el valor de ese campo en forma de String. Por ejemplo,

```
<%
String nombre = request.getParameter("nombre");
String apellido = request.getParameter("apellido");
%>
```

 UNIVERSIDAD Blas Pascal	INGENIERÍA EN INFORMÁTICA – PLAN 2014 PROGRAMACIÓN DISTRIBUIDA Y COMPONENTES – 9º CUATRIMESTRE	
	APUNTE DE JSP Y JAVA BEANS	VERSIÓN: 1.5 VIGENCIA: 09-04-2018

Cuando en el formulario de carga se incluya un campo que permita selección de valores múltiples, el objeto request cuenta con un método alternativo **getParameterValues()**, el cual toma como parámetro el nombre del campo de múltiples valores que se quiere acceder, y retorna un arreglo de cadenas con los valores de ese campo. Por ejemplo,

```
<%
String hobbies[] = request.getParameterValues("hobbies");
%>
```

Objetos implícitos

Toda página JSP contiene un conjunto de objetos implícitamente creados, y listos para usar en distintas tareas. El objeto **request** es uno de ellos, pero hay algunos más:

Objeto	Significado y uso
application	Este objeto de la clase javax.servlet.ServletContext representa el contenedor en el cual se ejecuta la JSP.
config	Este objeto de la clase javax.servlet.ServletConfig representa la opciones de configuración de la JSP.
exception	Este objeto de la clase java.lang.Throwable representa la excepción que es pasada a la página de error JSP. Sólo está disponible en una página JSP de error.
out	Este objeto de la clase javax.servlet.jsp.JspWriter escribe texto como parte de la respuesta a un requerimiento. Se usa implícitamente con expresiones y acciones JSP que insertan cadenas en una respuesta.
page	Este objeto de la clase java.lang.Object representa la referencia this para la instancia actual de la JSP.
pageContext	Este objeto de la clase javax.servlet.jsp.PageContext oculta los detalles de implementación del Servlet generado y del contenedor JSP, y provee a los programadores acceso a los objetos implícitos enunciados en esta tabla.
response	Este objeto representa la respuesta al cliente. Normalmente es instancia de una clase que implementa javax.servlet.http.HttpServletResponse. Si se usa un protocolo diferente de HTTP, el objeto es instancia de una clase que implementa javax.servlet.ServletResponse.
request	Este objeto representa la petición del cliente. Normalmente es instancia de una clase que implementa javax.servlet.http.HttpServletRequest. Si se usa un protocolo diferente de HTTP, el objeto es instancia de una subclase de javax.servlet.ServletRequest.
session	Este objeto de la clase javax.servlet.http.HttpSession representa la información de la sesión del cliente si tal sesión ha sido creada.

Acciones estándar

Como se mencionó anteriormente, además de la etiquetas **<% y %>** existe un formato XML de etiquetas JSP, de la forma **<jsp:etiqueta>** y **</jsp:etiqueta>**. Un uso común de estas formas de etiquetas es la introducción de acciones estándar.

Estas acciones proveen a los desarrolladores de una forma flexible de acceder a las tareas más comunes que pueden ser necesarias en un programa JSP, como podría ser incluir recursos, reenviar peticiones a otros recursos o interactuar con JavaBeans. Estas acciones son procesadas en tiempo de petición (cuando la petición se realiza). Las acciones estándar son las siguientes:

Acción	Descripción
--------	-------------

 UNIVERSIDAD Blas Pascal	INGENIERÍA EN INFORMÁTICA – PLAN 2014 PROGRAMACIÓN DISTRIBUIDA Y COMPONENTES – 9º CUATRIMESTRE	
	APUNTE DE JSP Y JAVA BEANS	VERSIÓN: 1.5 VIGENCIA: 09-04-2018

<jsp:include>	Incluye dinámicamente otros recursos en una JSP. A medida que la página se ejecuta, los recursos referidos se incluyen y se procesan.
<jsp:forward>	Redirecciona una petición hacia otra JSP, servlet o página estática. Esta acción finaliza la ejecución de la JSP actual.
<jsp:plugin>	Permite que un componente plugin se añada a la página en forma de objeto específico o elemento HTML embebido.
<jsp:param>	Se usa junto con las acciones include, forward y plugin para especificar pares adicionales de la forma nombre/valor para ser usados por estas acciones.
<jsp:useBean>	Indica que la JSP usa una instancia JavaBean. Especifica el ámbito del bean y le asigna un ID que puede ser usado por componentes que manipulen al bean.
<jsp:setProperty>	Configura una propiedad dentro de la instancia JavaBean especificada.
<jsp:getProperty>	Obtiene una propiedad desde la instancia JavaBean especificada y convierte el resultado a una cadena para ser desplegada en la respuesta al cliente.

 UNIVERSIDAD Blas Pascal	INGENIERÍA EN INFORMÁTICA – PLAN 2014 PROGRAMACIÓN DISTRIBUIDA Y COMPONENTES – 9º CUATRIMESTRE	
	APUNTE DE JSP Y JAVA BEANS	VERSIÓN: 1.5 VIGENCIA: 09-04-2018

JavaBeans

Un JavaBean es un modelo de componente de software reusable en lenguaje Java, que puede ser manipulado visualmente por una herramienta de programación. Cualquier clase Java que adhiere a ciertas convenciones en lo relativo a las propiedades, métodos y eventos puede ser un JavaBean.

Los componentes JavaBean son conocidos como beans.

Un bean es un componente, portable, visible o no, autocontenido y escrito en Java que cumple con las recomendaciones de la especificación JavaBean, en lo referente a los protocolos de comunicación y configuración del objeto generado.

Un bean es una clase Java que sigue determinadas convenciones, de manera que pueda ser empleada como un componente por herramientas de desarrollo, para la construcción de aplicaciones.

Características

1. **Introspección**: Las herramientas de construcción de beans pueden descubrir las características de un bean (propiedades, métodos y eventos) y analizar cómo trabaja. La información de una clase, tales como nombre de los métodos, parámetros y tipos de retorno, pueden ser descubiertas por otras clases.
2. **Propiedades**: Una propiedad es un atributo de un bean que puede ser leído o escrito por el cliente del bean, a través de métodos regulares cuyos nombres responden a los lineamientos de construcción de JavaBeans.
3. **Customización**: Permite al desarrollador establecer y personalizar nuevas propiedades que gobiernan el comportamiento del bean.
4. **Eventos**: Permite a un bean comunicarse con otros beans.
5. **Persistencia**: Habilita al bean a guardar y recuperar su estado. Debe implementar la interface `java.io.Serializable`

Convenciones

1. Un JavaBean es una clase cuyo constructor no tiene argumentos.
2. No puede tener variables de instancias públicas, también conocidos como atributos de la clase.
3. Los valores persistentes deberán ser accedidos a través de los métodos getter y setter, conocidos como métodos de acceso.
4. Los nombres de los métodos getter o setter, se componen de la palabra get o set, más el nombre de la propiedad con su primera letra en mayúscula.
5. Un método regular getter no tiene parámetros y retorna un valor del tipo de la propiedad.
6. Un método setter tiene un único parámetro que es del tipo de la propiedad y tiene un retorno de tipo void.
7. Una propiedad legible debe tener un método getter.
8. Una propiedad escribible debe tener un método setter.
9. Dependiendo de la combinación de métodos getter y setter, una propiedad es solo lectura, solo escritura o lectura y escritura.
10. Una propiedad solo lectura no necesariamente debe coincidir con una variable de instancia. Puede ser un valor computado resultado de combinar varias variables de instancia.
11. El tipo de una propiedad puede ser una clase, una interface o un tipo primitivo.
12. Los beans pueden tener propiedades de múltiples valores representados por un vector de cualquier tipo. Esto es denominado una "propiedad indexada".
13. Dos tipos de métodos de acceso pueden tener una propiedad indexada:
 - a. Métodos de lectura y escritura de todo el vector.
 - b. Métodos de lectura y escritura de un elemento del vector.

 UNIVERSIDAD Blas Pascal	INGENIERÍA EN INFORMÁTICA – PLAN 2014 PROGRAMACIÓN DISTRIBUIDA Y COMPONENTES – 9º CUATRIMESTRE	
	APUNTE DE JSP Y JAVA BEANS	VERSIÓN: 1.5 VIGENCIA: 09-04-2018

14. En una propiedad booleana se puede emplear el método getter regular. Pero la recomendación es usar la palabra `is` combinada con el nombre de la propiedad con su primera letra en mayúscula.
15. El método setter de una propiedad booleana sigue el patrón general.

Veamos algunos ejemplos,

```
package ar.edu.ubp.pdc.beans;

public class ProductoBean implements java.io.Serializable {

    private String nomProducto;
    private int cantidad;
    private float precioUnitario;

    public ProductoBean() {
    }

    public int getCantidad() {
        return cantidad;
    }

    public String getNomProducto() {
        return nomProducto;
    }

    public float getPrecioUnitario() {
        return precioUnitario;
    }

    public float getPrecioTotal() {
        return precioUnitario * cantidad;
    }

    public void setCantidad(int cantidad) {
        this.cantidad = cantidad;
    }

    public void setNomProducto(String nomProducto) {
        this.nomProducto = nomProducto;
    }

    public void setPrecioUnitario(float precioUnitario) {
        this.precioUnitario = precioUnitario;
    }

}
```

Tipos de beans

Existen dos tipos:

- (1) Value beans: Encapsula toda la información acerca de una entidad, tal como un usuario, producto, etc.

 UNIVERSIDAD Blas Pascal	INGENIERÍA EN INFORMÁTICA – PLAN 2014 PROGRAMACIÓN DISTRIBUIDA Y COMPONENTES – 9º CUATRIMESTRE	
	APUNTE DE JSP Y JAVA BEANS	VERSIÓN: 1.5 VIGENCIA: 09-04-2018

- (2) Utility beans: Efectúa alguna acción, tal como guardar información en la base de datos o enviar un e-mail. "Utility beans" pueden emplear "Value beans" como entrada o producirlos como resultado de una acción.

Empleo de beans

Para declarar el uso de una instancia bean en una página JSP se usa la directiva:

Sintaxis 1

<jsp:useBean id="name" scope="scope" Bean-Specification/>

Sintaxis 2

<jsp:useBean id="name" scope="scope" Bean-Specification>
Creation-Body
</jsp:useBean>

id: Nombre con el cual se referenciará el bean dentro del JSP. Atributo obligatorio.

scope: Ámbito de visibilidad del bean, entre ellos:

Ámbito	Duración
page	El bean solo será válido dentro de la página JSP y será regenerado en cada nueva petición.
request	El bean será válido a lo largo de la petición y estará disponible para incluirse o reenviarse a otras páginas JSP.
session	El bean se asocia a una sesión en particular, la cual es responsable de su creación. El bean será válido mientras la sesión exista.
application	El bean es común a todas las sesiones y será válido hasta que la aplicación Web finalice.

Bean-Specification: Atributos adicionales que sirven para declarar el bean. Debe considerarse que el uso de estos es mutuamente excluyente, es decir, que existen determinadas combinaciones a respetar. Entre ellos encontramos:

Atributo	Descripción
class="className"	Clase Java que implementa al objeto.
type="typeName"	Especifica el tipo de la variable que identifica al bean. Este debe corresponderse con el nombre de la clase Java o superclase o interfaz que implementa la clase Java.
beanName="beanName"	El nombre del bean en caso de que esté serializado (persistencia), como lo suministraríamos en el método <code>java.beans.Beans.instantiate()</code> .

Los atributos antes especificados deben aparecer en las siguientes combinaciones:

- class
- type
- class y type
- beanName y type

Creation-Body: Se ejecuta solo si el bean no existe y sirve para inicializar el mismo. El bean solo es creado si "no es encontrado dentro del scope establecido".

 UNIVERSIDAD Blas Pascal	INGENIERÍA EN INFORMÁTICA – PLAN 2014 PROGRAMACIÓN DISTRIBUIDA Y COMPONENTES – 9º CUATRIMESTRE	
	APUNTE DE JSP Y JAVA BEANS	VERSIÓN: 1.5 VIGENCIA: 09-04-2018

Para establecer un valor a una propiedad del bean, se usa:

<jsp:setProperty name="beanId" property="propertyName" Property-Specification />

name: Nombre con el cual se identifica el bean en la página JSP.

property: Nombre de la propiedad del bean a establecer su valor.

Property-Specification: Atributos adicionales cuyo uso es mutuamente excluyente. Entre ellos encontramos:

Atributo	Descripción
value="value"	Valor a establecer.
param="param"	Nombre del parámetro disponible en el HttpServletRequest cuyo valor será asignado a la propiedad.

Si no se especifica ninguno de estos atributos, se usa por defecto "param"

Para obtener el valor actual de una propiedad de un bean, se usa:

<jsp:getProperty name="beanId" property="propertyName" />

name: Nombre con el cual se identifica el bean en la página JSP.

property: Nombre de la propiedad del bean a establecer su valor.